TIGER CS TICKETING SYSTEM

Application and Solution Architecture

Built on ASP.NET Core, .NET 10 (LTS)

Internal Pilot Architecture

Business Objective

1

Centralize

All customer-service requests captured in a single system of record

2

Integrate

Phone (Genesys), CRM, and internal departments into one workflow

3

Track

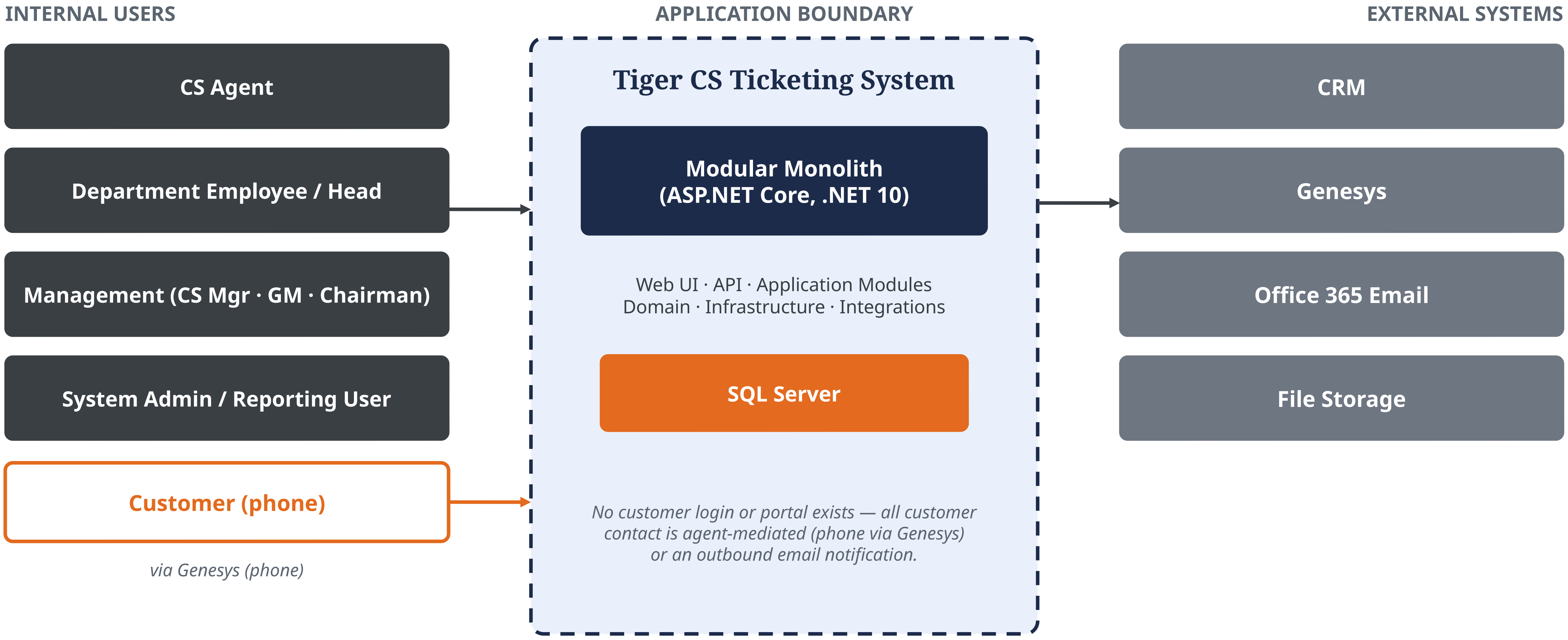
Ownership, SLA due dates, escalation, and resolution for every ticket

4

Provide

Auditability and management visibility across the full ticket lifecycle

System Context Architecture



Application Architecture

Modular monolith — one deployable unit, clean internal module boundaries

Web UI

ASP.NET Core API

APPLICATION MODULES

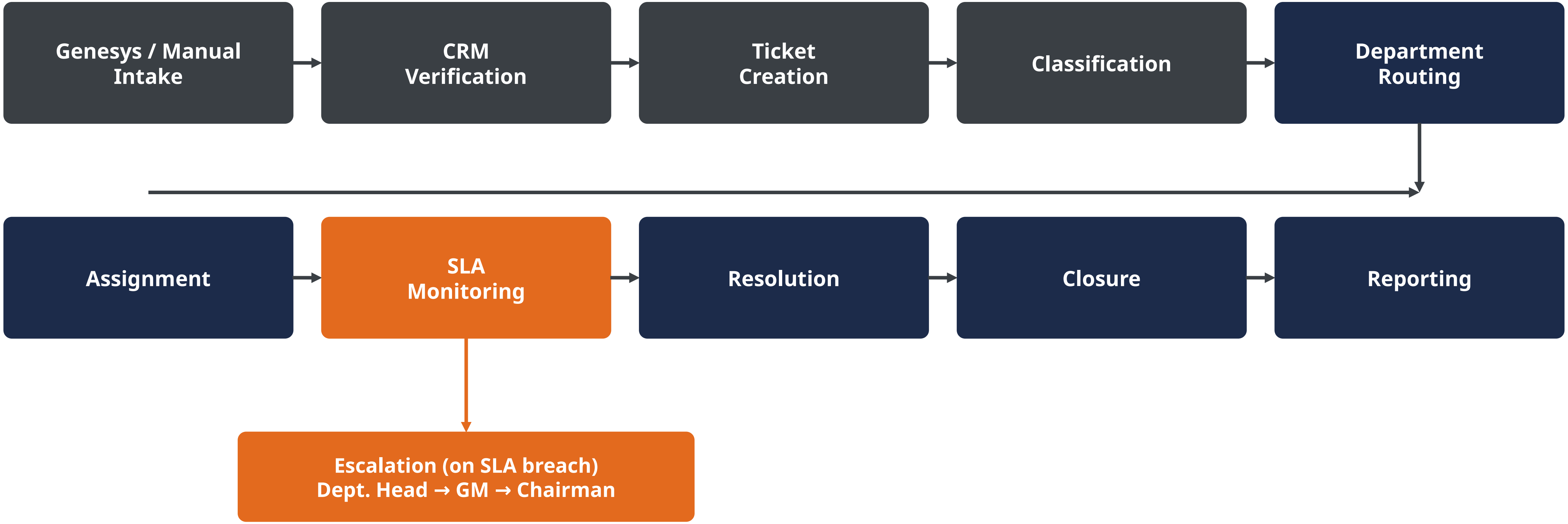
Identity & Access	CRM Verification	Intake	Ticketing	SLA & Escalation
Genesys Integration	Notifications	Attachments	Audit	Reporting

Domain

Infrastructure | Integrations (CRM · Genesys · Email · Storage)

SQL Server

Main Ticket Flow



Escalation runs alongside the main flow whenever a ticket's SLA is breached, independent of ticket status.

Integration Architecture

CRM

Source of truth for
units and customers

Tiger CS Ticketing System

Genesys

Supplies call and
conversation data

Office 365 Email

Sends acknowledgement
to customer

File Storage

Holds ticket
attachments

✓ Outbox & Idempotency

Every integration effect is written transactionally and dispatched safely — protecting against duplicate or lost processing on retries and outages.

⚡ Feature Flags

Each integration (notably Genesys) is activated behind a feature flag — allowing safe, staged rollout without a code redeploy.

Security Architecture

ASP.NET Core Identity

Staff authentication — no customer accounts exist

JWT Authentication

Issued on login, validated on every API request

Role- & Department-Based Authorization

Every action checked server-side against policy

Nine Approved Roles

From CS Agent through Chairman/CEO and System Administrator

Account Deactivation

Validated on every request, not only at login

Full Audit Trail

Append-only — no update or delete path

Secrets & Transport

Secrets kept outside source control; HTTPS everywhere

Least-Privilege Access

No production credentials ever stored in GitHub

Deployment Architecture

Target: Tiger-approved hosting environment (final hosting provider to be confirmed by Tiger IT)

Development	UAT / Pilot	Production
Web Server / Secure Gateway (IIS or Nginx)	Web Server / Secure Gateway (IIS or Nginx)	Web Server / Secure Gateway (IIS or Nginx)
ASP.NET Core Web / API	ASP.NET Core Web / API	ASP.NET Core Web / API
Background Jobs	Background Jobs	Background Jobs
SQL Server	SQL Server	SQL Server
File Storage	File Storage	File Storage
Monitoring / Logging	Monitoring / Logging	Monitoring / Logging
Backup & Recovery	Backup & Recovery	Backup & Recovery

Delivery Scope and Roadmap

Current
.NET 10 solution foundation completed

In Progress
Identity and Access

Next
CRM Verification & Ticket Creation

Then
Workflow, SLA/Escalation, Genesys, Email, UI, UAT

Three-Week Internal Pilot

A functional version will be delivered for internal testing within three weeks, developed by one developer with AI-assisted development. Following successful testing and management approval, a separate production-readiness phase will prepare the system for official go-live.

P

Pilot (This Delivery)

A functional internal proof of the approved architecture, deployed to a non-production environment for evaluation.

R

Production Ready

A separate, later milestone — its own scope, hardening, and go-live approval, not implied by pilot completion.

Key Risks and Management Decisions

- ! CRM and Genesys integration contract availability
- ! Schedule pressure from a single-developer delivery model
- ! UAT participation from CS and department staff
- ! SQL Server and target environment readiness
- ! Independent security review before pilot go-live

Production go-live requires separate, explicit management approval.